

Engenharia de Software – PUC Campinas

Algoritmos e Linguagens de Programação

Prof. Dr. Paulo Cesar de Alvarenga Lucci

Funções

Funções

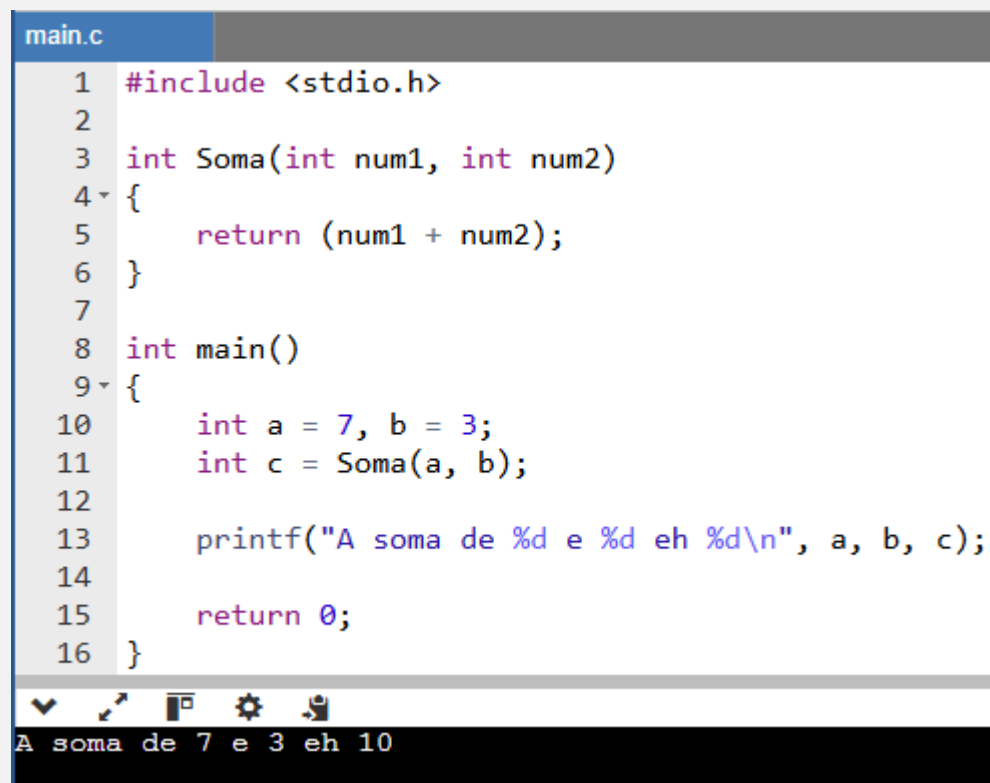
- O QUE SÃO?

- **Funções** são blocos de código, **externos ao main()**, que executam tarefas específicas.
- Elas permitem dividir um programa em partes menores, facilitando a leitura, a manutenção e a reutilização de código.
- **Vantagens de usar funções:**
 - Organização do código.
 - Reutilização (evita repetição).
 - Facilita testes e depuração.
 - Modularização do programa.
 - Reduz complexidade.

Funções

○ SINTAXE BÁSICA DE UMA FUNÇÃO:

```
tipo_retorno nome_da_funcao(tipo_parâmetro(s) parâmetro(s))  
{  
    // bloco de código  
    return alguma_coisa;  
}
```



```
main.c  
1  #include <stdio.h>  
2  
3  int Soma(int num1, int num2)  
4  {  
5      return (num1 + num2);  
6  }  
7  
8  int main()  
9  {  
10     int a = 7, b = 3;  
11     int c = Soma(a, b);  
12  
13     printf("A soma de %d e %d eh %d\n", a, b, c);  
14  
15     return 0;  
16 }
```

A soma de 7 e 3 eh 10

Funções

○ FLUXO DE EXECUÇÃO

- Quando uma função é chamada em um determinado escopo, o fluxo da execução é desviado para o escopo da função.
- Quando a função é finalizada, o fluxo volta para o escopo que a chamou.

```
main.c
1  #include <stdio.h>
2
3  int Soma(int num1, int num2)
4  {
5      return (num1 + num2);
6  }
7
8  int main()
9  {
10     int a = 7, b = 3;
11     int c = Soma(a, b);
12
13     printf("A soma de %d e %d eh %d\n", a, b, c);
14
15     return 0;
16 }
```

A soma de 7 e 3 eh 10

...Program finished with exit code 0
Press ENTER to exit console.

Funções

○ FLUXO DE EXECUÇÃO

```
main.c
1  #include <stdio.h>
2
3  int Soma(int num1, int num2)
4  {
5      return (num1 + num2);
6  }
7
8  int main()
9  {
10     int a = 7, b = 3;
11     int c = Soma(a, b);
12
13     printf("A soma de %d e %d eh %d\n", a, b, c);
14
15     return 0;
16 }
```

A soma de 7 e 3 eh 10

...Program finished with exit code 0
Press ENTER to exit console.

Funções

○ FLUXO DE EXECUÇÃO

```
main.c
1  #include <stdio.h>
2
3  int Soma(int num1, int num2)
4  {
5      → return (num1 + num2);
6  }
7
8  int main()
9  {
10     int a = 7, b = 3;
11     int c = Soma(a, b);
12
13     printf("A soma de %d e %d eh %d\n", a, b, c);
14
15     return 0;
16 }
```

A soma de 7 e 3 eh 10

...Program finished with exit code 0
Press ENTER to exit console.

Funções

○ FLUXO DE EXECUÇÃO

```
main.c
1  #include <stdio.h>
2
3  int Soma(int num1, int num2)
4  {
5      return (num1 + num2);
6  }
7
8  int main()
9  {
10     int a = 7, b = 3;
11     int c = Soma(a, b);
12
13     printf("A soma de %d e %d eh %d\n", a, b, c);
14
15     return 0;
16 }
```

A soma de 7 e 3 eh 10

...Program finished with exit code 0
Press ENTER to exit console.

Funções

○ FLUXO DE EXECUÇÃO

```
main.c
1  #include <stdio.h>
2
3  int Soma(int num1, int num2)
4  {
5      return (num1 + num2);
6  }
7
8  int main()
9  {
10     int a = 7, b = 3;
11     int c = Soma(a, b);
12
13     printf("A soma de %d e %d eh %d\n", a, b, c);
14
15     return 0;
16 }
```

A soma de 7 e 3 eh 10

...Program finished with exit code 0
Press ENTER to exit console.

Funções

- RETORNO DE FUNÇÃO:

- **Tipo de retorno:** Explicita o **tipo** retornado pela função através do **return alguma_coisa**.
- Nem sempre uma função retorna algo.
 - Neste caso o **tipo** é **void** e a função não tem **return alguma_coisa**.
 - Pode ter somente "**return;**" para sair da função.

```
main.c
1 #include <stdio.h>
2
3 void Maior(int num1, int num2)
4 {
5     if (num1 > num2) printf("O maior eh %d\n", num1);
6     else if (num2 > num1) printf("O maior eh %d\n", num2);
7     else printf("Os numeros fornecidos sao iguais\n");
8 }
9
10 int main()
11 {
12     int a = 7, b = 3;
13     Maior(a, b);
14
15     return 0;
16 }
```

O maior eh 7

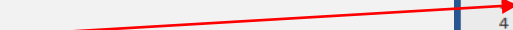
```
main.c
1 #include <stdio.h>
2
3 void AvisoDivisaoPorZero()
4 {
5     printf("Encontrada divisao por zero!!!\n");
6 }
7
8 void PrintDivisao(double a, double b)
9 {
10     if (b == 0)
11     {
12         AvisoDivisaoPorZero();
13         return;
14     }
15
16     double c = a / b;
17     printf("A divisao de %lf por %lf eh %lf\n", a, b, c);
18 }
19
20 int main()
21 {
22     double a = 7, b = 0;
23
24     PrintDivisao(a, b);
25
26     return 0;
27 }
```

Encontrada divisao por zero!!!

Funções

○ PARÂMETROS DE FUNÇÃO:

- São valores enviados para a função no momento da chamada.
- Os parâmetros podem ser passados de duas maneiras:
 - **Por valor:** Uma cópia do valor do argumento é passada para a função.
 - **Por referência:** Um endereço de memória do argumento é passado, permitindo que a função modifique seu valor original.
- Nem sempre uma função recebe parâmetros.
- Exemplos são funções que operaram em **atributos** da classe, emissão de avisos etc.



```
main.c
1  #include <stdio.h>
2
3  void AvisoDivisaoPorZero()
4  {
5      printf("Encontrada divisao por zero!!!\n");
6  }
7
8  void PrintDivisao(double a, double b)
9  {
10     if (b == 0)
11     {
12         AvisoDivisaoPorZero();
13         return;
14     }
15
16     double c = a / b;
17     printf("A divisao de %lf por %lf eh %lf\n", a, b, c);
18 }
19
20 int main()
21 {
22     double a = 7, b = 0;
23
24     PrintDivisao(a, b);
25
26     return 0;
27 }
```

Encontrada divisao por zero!!!

Funções

○ PARÂMETROS POR VALOR:

- Quando um parâmetro é passado por **valor**, uma **cópia** do valor da variável é criada dentro da função.
- **Alterações** feitas nesse parâmetro **dentro** da função **não afetam** a **variável original** (fora da função).
- É o comportamento padrão em C e a maioria das linguagens de programação.
- Exemplo:

```
main.c
1  #include <stdio.h>
2
3  void dobrar_valor(int x)
4  {
5      x = x * 2;
6      printf("Dentro da função: %d\n", x);
7  }
8
9  int main()
10 {
11     int numero = 5;
12     dobrar_valor(numero); // Passa uma cópia do valor de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 5
14     return 0;
15 }
```

Dentro da função: 10
Fora da função: 5

...Program finished with exit code 0
Press ENTER to exit console.

Funções

5
numero
Endereço AA

```
main.c
1  #include <stdio.h>
2
3  void dobrar_valor(int x)
4  {
5      x = x * 2;
6      printf("Dentro da função: %d\n", x);
7  }
8
9  int main()
10 {
11     int numero = 5;
12     dobrar_valor(numero); // Passa uma cópia do valor de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 5
14     return 0;
15 }
```

```
Dentro da função: 10
Fora da função: 5
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Funções

5

numero

Endereço AA

main.c

```
1  #include <stdio.h>
2
3  void dobrar_valor(int x)
4  {
5      x = x * 2;
6      printf("Dentro da função: %d\n", x);
7  }
8
9  int main()
10 {
11     int numero = 5;
12     dobrar_valor(numero); // Passa uma cópia do valor de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 5
14     return 0;
15 }
```

Dentro da função: 10
Fora da função: 5

...Program finished with exit code 0
Press ENTER to exit console.

Funções

5
numero
Endereço AA

5
x
Endereço BB

```
main.c
1  #include <stdio.h>
2
3  void dobrar_valor(int x)
4  {
5      x = x * 2;
6      printf("Dentro da função: %d\n", x);
7  }
8
9  int main()
10 {
11     int numero = 5;
12     dobrar_valor(numero); // Passa uma cópia do valor de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 5
14     return 0;
15 }
```

```
Dentro da função: 10
Fora da função: 5
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Funções

5
numero
Endereço AA

10
x
Endereço BB

```
main.c
1  #include <stdio.h>
2
3  void dobrar_valor(int x)
4  {
5      x = x * 2;
6      printf("Dentro da função: %d\n", x);
7  }
8
9  int main()
10 {
11     int numero = 5;
12     dobrar_valor(numero); // Passa uma cópia do valor de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 5
14     return 0;
15 }
```

```
Dentro da função: 10
Fora da função: 5
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Funções

5
numero
Endereço AA

10
x
Endereço BB

```
main.c
1  #include <stdio.h>
2
3  void dobrar_valor(int x)
4  {
5      x = x * 2;
6      printf("Dentro da função: %d\n", x);
7  }
8
9  int main()
10 {
11     int numero = 5;
12     dobrar_valor(numero); // Passa uma cópia do valor de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 5
14     return 0;
15 }
```

```
Dentro da função: 10
Fora da função: 5

...Program finished with exit code 0
Press ENTER to exit console.
```

Funções

5
numero
Endereço AA

```
main.c
1  #include <stdio.h>
2
3  void dobrar_valor(int x)
4  {
5      x = x * 2;
6      printf("Dentro da função: %d\n", x);
7  }
8
9  int main()
10 {
11     int numero = 5;
12     dobrar_valor(numero); // Passa uma cópia do valor de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 5
14     return 0;
15 }
```

```
Dentro da função: 10
Fora da função: 5

...Program finished with exit code 0
Press ENTER to exit console.
```

Funções

○ PARÂMETROS POR REFERÊNCIA:

- Na passagem por **referência**, o **endereço** da variável é passado para a função.
- Isso permite que a função **acesse e modifique o valor original da variável**.
- Em C, a passagem por referência é implementada através de **ponteiros (*)**.

main.c

```
1  #include <stdio.h>
2
3  void dobrar_valor_ref(int *x)
4  {
5      *x = (*x) * 2; // Modifica o valor no endereço apontado por 'x'
6      printf("Dentro da função: %d\n", *x);
7  }
8
9  int main()
10 {
11     int numero = 5;
12     dobrar_valor_ref(&numero); // Passa o endereço de memória de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 10
14     return 0;
15 }
```

Dentro da função: 10

Fora da função: 10

...Program finished with exit code 0
Press ENTER to exit console.

Funções

5
numero

Endereço AA

```
main.c
1  #include <stdio.h>
2
3  void dobrar_valor_ref(int *x)
4  {
5      *x = (*x) * 2; // Modifica o valor no endereço apontado por 'x'
6      printf("Dentro da função: %d\n", *x);
7  }
8
9  int main()
10 {
11     int numero = 5;
12     dobrar_valor_ref(&numero); // Passa o endereço de memória de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 10
14     return 0;
15 }
```

Dentro da função: 10

Fora da função: 10

...Program finished with exit code 0

Press ENTER to exit console.

Funções

5
numero

Endereço AA

main.c

```
1 #include <stdio.h>
2
3 void dobrar_valor_ref(int *x)
4 {
5     *x = (*x) * 2; // Modifica o valor no endereço apontado por 'x'
6     printf("Dentro da função: %d\n", *x);
7 }
8
9 int main()
10 {
11     int numero = 5;
12     dobrar_valor_ref(&numero); // Passa o endereço de memória de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 10
14     return 0;
15 }
```

Dentro da função: 10

Fora da função: 10

...Program finished with exit code 0

Press ENTER to exit console.

Funções

5
x

Endereço AA

```
main.c
1  #include <stdio.h>
2
3  void dobrar_valor_ref(int *x)
4  {
5      *x = (*x) * 2; // Modifica o valor no endereço apontado por 'x'
6      printf("Dentro da função: %d\n", *x);
7  }
8
9  int main()
10 {
11     int numero = 5;
12     dobrar_valor_ref(&numero); // Passa o endereço de memória de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 10
14     return 0;
15 }
```

Dentro da função: 10

Fora da função: 10

...Program finished with exit code 0

Press ENTER to exit console.

Funções

10
x

Endereço AA

main.c

```
1 #include <stdio.h>
2
3 void dobrar_valor_ref(int *x)
4 {
5     *x = (*x) * 2; // Modifica o valor no endereço apontado por 'x'
6     printf("Dentro da função: %d\n", *x);
7 }
8
9 int main()
10 {
11     int numero = 5;
12     dobrar_valor_ref(&numero); // Passa o endereço de memória de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 10
14     return 0;
15 }
```

Dentro da função: 10

Fora da função: 10

...Program finished with exit code 0

Press ENTER to exit console.

Funções

10
x

Endereço AA

main.c

```
1 #include <stdio.h>
2
3 void dobrar_valor_ref(int *x)
4 {
5     *x = (*x) * 2; // Modifica o valor no endereço apontado por 'x'
6     printf("Dentro da função: %d\n", *x);
7 }
8
9 int main()
10 {
11     int numero = 5;
12     dobrar_valor_ref(&numero); // Passa o endereço de memória de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 10
14     return 0;
15 }
```

```
Dentro da função: 10
Fora da função: 10
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Funções

10
numero
Endereço AA

```
main.c
1  #include <stdio.h>
2
3  void dobrar_valor_ref(int *x)
4  {
5      *x = (*x) * 2; // Modifica o valor no endereço apontado por 'x'
6      printf("Dentro da função: %d\n", *x);
7  }
8
9  int main()
10 {
11     int numero = 5;
12     dobrar_valor_ref(&numero); // Passa o endereço de memória de 'numero'
13     printf("Fora da função: %d\n", numero); // Imprime 10
14     return 0;
15 }
```

```
Dentro da função: 10
Fora da função: 10
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

Funções

- UTILIZAÇÕES DE PARÂMETROS POR REFERÊNCIA:

- Parâmetros por referência permite dois usos importantes:

- Fazer valer modificações valerem fora da função:

- Ideal para quando queremos que a função altere variáveis originais.

- Evitar cópia desnecessária:

- Mesmo que não se deseje alterar o valor, passar grandes estruturas de dados ou objetos por referência evita o custo de cópia na memória, otimizando desempenho e uso de recursos.

Funções

○ EVITAR CÓPIA DESNECESSÁRIA

- Imagine a estrutura de dados ao lado que representa uma pessoa física (PF).
- Se uma função espera receber por parâmetro uma variável do tipo PF **por valor**, dentro da função irá uma **cópia** de toda a estrutura de dados.
- Se uma função espera receber por parâmetro uma variável do tipo PF **por referência**, **não é feita a cópia** uma vez que **dentro da função** teremos a **própria variável**.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <locale.h>
4 #include <string.h>
5
6 struct PF
7 {
8     char nome[50];
9
10    char rua_res[50];
11    int numero_res;
12    int cep_res;
13    char cidade_res[20];
14
15    char cidade_nascimento[20];
16    char estado_nascimento[3];
17    char sexo;
18
19    long cpf;
20    int rg;
21
22    int dia_nascimento;
23    int mes_nascimento;
24    int ano_nascimento;
25 };
```

Funções

○ POSSIBILIDADES DE RETORNOS DE FUNÇÕES:

Retorno principal (**return** alguma_coisa)

```
main.c
1  #include <stdio.h>
2
3  double Soma(double num1, double num2)
4  {
5      double val = num1 + num2;
6      return val;
7  }
8
9  int main()
10 {
11     double a = 7.54;
12     double b = 3.28;
13     double c = Soma(a, b);
14
15     printf("A soma dos números fornecidos é %f", c);
16
17     return 0;
18 }
```

A soma dos números fornecidos é 10.820000

Retorno por parâmetros por referência

```
main.c
1  #include <stdio.h>
2
3  void MinMax(double num1, double num2, double num3, double *min, double *max)
4  {
5      *min = num1;
6      if (num2 < *min) *min = num2;
7      if (num3 < *min) *min = num3;
8
9      *max = num1;
10     if (num2 > *max) *max = num2;
11     if (num3 > *max) *max = num3;
12 }
13
14 int main()
15 {
16     double a = 17.54;
17     double b = 3.28;
18     double c = 9.33;
19
20     double minNum, maxNum;
21     MinMax(a, b, c, &minNum, &maxNum);
22
23     printf("O mínimo entre os 3 números é %f\n", minNum);
24     printf("O máximo entre os 3 números é %f\n", maxNum);
25
26     return 0;
27 }
```

O mínimo entre os 3 números é 3.280000
O máximo entre os 3 números é 17.540000

Vetores e Matrizes como parâmetros

Funções

○ VETORES E MATRIZES COMO PARÂMETROS

- Funções podem receber vetores e matrizes como argumentos.
- Isso permite tornar o código mais modular e organizado.
- A passagem é feita por referência sem a necessidade de ponteiro.
- Notas de **vetor** como parâmetro:
 - O tamanho deve ser passado como **parâmetro**.
- Notas de **matriz** como parâmetro:
 - É **obrigatório** informar o número de **colunas**.
 - O número de **linhas** deve ser passado como **parâmetro**.

Por serem passados por referência, alterações dentro da função afetam o original.

```
main.c
1  #include <stdio.h>
2
3  void ImprimirVetor(int v[], int tamanho)
4  {
5      for (int i = 0; i < tamanho; i++)
6      {
7          printf("%d\n", v[i]);
8      }
9      printf("\n");
10 }
11
12 void ImprimirMatriz(int m[][3], int linhas) {
13     for (int i = 0; i < linhas; i++)
14     {
15         for (int j = 0; j < 3; j++)
16         {
17             printf("%d ", m[i][j]);
18         }
19         printf("\n");
20     }
21 }
22
23 int main()
24 {
25     int vec[5] = { 7, 1, 9, 3, 4 };
26     printf("Vetor:\n");
27     ImprimirVetor(vec, 5);
28
29     int matr[2][3] = { {8, 6, 7}, {1, 3, 0} };
30     printf("Matriz:\n");
31     ImprimirMatriz(matr, 2);
32
33     return 0;
34 }
```

Vetor:

7
1
9
3
4

Matriz:

8 6 7
1 3 0

Funções

○ VETORES E MATRIZES COMO PARÂMETROS

- Compiladores a partir da versão C99 contam com o "VLA" (Variable Length Array) como parâmetro de função, ou seja, um array cujo tamanho depende de parâmetros passados à função.
- Com isso é possível escrever funções com vetores e matrizes da seguinte forma:

```
void PrintMatrix(int nrows, int ncols, int M[nrows][ncols])
{
    for (int i = 0; i < nrows; i++)
    {
        for (int j = 0; j < ncols; j++)
        {
            printf("%d\t", M[i][j]);
        }
        printf("\n");
    }
}
```

A partir do C99, a especificação permite essa forma porque o compilador entende que `nrows` e `ncols` são conhecidos no momento da chamada da função e, portanto, pode usar isso para interpretar `M[nrows][ncols]`.

Funções

- ESCOPO DE VARIÁVEIS EM C

- O escopo define onde uma variável pode ser acessada dentro do programa.

- Variáveis Locais:

- Declaradas dentro de escopos de funções ou blocos.
 - Só existem durante a execução da função/bloco.
 - Não podem ser acessadas fora do local onde foram declaradas.

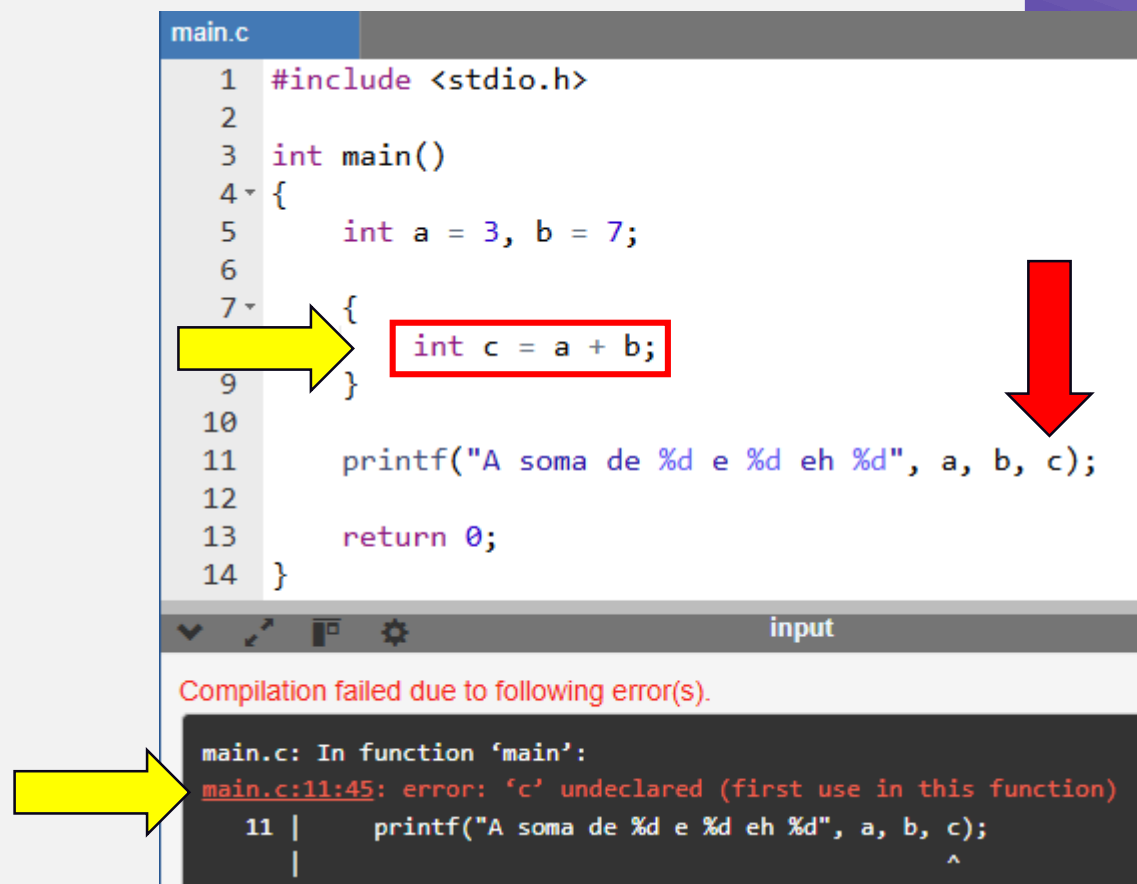
- Variáveis Globais:

- Declaradas fora de qualquer escopo, geralmente no início do programa.
 - Podem ser acessadas por todas as funções do programa.
 - Permanecem na memória durante toda a execução.

Funções

○ ESCOPO DE VARIÁVEIS EM C

- No exemplo, a variável **c** foi declarada dentro de um escopo e, portanto, só existe dentro deste escopo (variável **local** ao seu escopo).
- Percebam o erro de compilação ao tentar utilizar a variável **c** fora do escopo em que foi criada.



```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int a = 3, b = 7;
6
7      {
9          int c = a + b;
10
11         printf("A soma de %d e %d eh %d", a, b, c);
12
13         return 0;
14     }
```

Compilation failed due to following error(s).

```
main.c: In function 'main':
main.c:11:45: error: 'c' undeclared (first use in this function)
11 |     printf("A soma de %d e %d eh %d", a, b, c);
    |                                             ^
```

Funções

○ ESCOPO DE VARIÁVEIS EM C

- Agora observamos a variável **c** sendo criada fora do escopo da soma.
- Variáveis locais são válidas no seu escopo e em escopos internos.
- A alteração da variável **c** no escopo interno foi válida!!!

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int a = 3, b = 7, c;
6
7      {
8          c = a + b;
9      }
10
11     printf("A soma de %d e %d eh %d", a, b, c);
12
13     return 0;
14 }
```

A soma de 3 e 7 eh 10

...Program finished with exit code 0
Press ENTER to exit console.

Funções

main.c

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int a = 10;
6
7     {
8         //a é reconhecido aqui!!!!
9         {
10             //a é reconhecido aqui!!!!
11             int b = 20;
12             {
13                 //a é reconhecido aqui!!!!
14                 //b é reconhecido aqui!!!!
15                 {
16                     int c = 30;
17                     //a é reconhecido aqui!!!!
18                     //b é reconhecido aqui!!!!
19                     //c é reconhecido aqui!!!!
20                 }
21                 //a é reconhecido aqui!!!!
22                 //b é reconhecido aqui!!!!
23                 //c NÃO é reconhecido aqui!!!!
24             }
25             //a é reconhecido aqui!!!!
26             //b é reconhecido aqui!!!!
27             //c NÃO é reconhecido aqui!!!!
28         }
29         //a é reconhecido aqui!!!!
30         //b NÃO é reconhecido aqui!!!!
31         //c NÃO é reconhecido aqui!!!!
32     }
33
34     //a é reconhecido aqui!!!!
35     //b NÃO é reconhecido aqui!!!!
36     //c NÃO é reconhecido aqui!!!!
37
38     return 0;
39 }
```

Funções

○ VARIÁVEIS LOCAIS E GLOBAIS

PI: variável global

variável local ao escopo da
função PerimetroCirculo

variável local ao escopo da
função AreaCirculo

variáveis locais ao
escopo do main()

```
main.c
1  #include <stdio.h>
2
3  const double PI = 3.1415926;
4
5  double PerimetroCirculo(double raio)
6  {
7      double perimetro = 2 * PI * raio;
8
9      return perimetro;
10 }
11
12 void PrintPerimetro(double r, double p)
13 {
14     printf("O perimetro do circulo de raio %lf eh %lf\n", r, p);
15 }
16
17 double AreaCirculo(double raio)
18 {
19     double area = PI * raio * raio;
20
21     return area;
22 }
23
24 void PrintArea(double r, double a)
25 {
26     printf("A area do circulo de raio %lf eh %lf\n", r, a);
27 }
28
29 int main()
30 {
31     double r = 5;
32     double perimetroCirc = PerimetroCirculo(r);
33     double areaCirc = AreaCirculo(r);
34
35     PrintPerimetro(r, perimetroCirc);
36     PrintArea(r, areaCirc);
37
38     return 0;
39 }
```

O perimetro do circulo de raio 5.000000 eh 31.415926
A area do circulo de raio 5.000000 eh 78.539815

Funções

- VARIÁVEIS GLOBAIS EM C: UTILIZAÇÕES, RECOMENDAÇÕES E CUIDADOS

- Utilizações:

- Compartilhar dados entre funções e arquivos, principalmente em programas simples ou quando o estado precisa ser acessado amplamente.

- Recomendações:

- Evite ao máximo o uso de variáveis globais!
 - Prefira passar dados como parâmetros para funções, mantendo variáveis locais sempre que possível.
 - Isso torna o código mais modular, legível e fácil de manter.

Exercícios

○ ESPECIFICIDADE DAS FUNÇÕES

- Neste momento do curso, o foco é o desenvolvimento de funções — **trechos de código bem definidos, reutilizáveis e independentes**.
- Cada função implementada deve ser **bem projetada**.
- Para isso, siga estas orientações:
 - Comece escrevendo a função. **Não comece pelo main**. O main serve apenas para testar a função — e não o contrário.
 - Pense na função como uma **"caixa-preta"**: ela deve receber somente os parâmetros **estritamente necessários** para executar sua tarefa e retornar um resultado (se for o caso), **sem depender de variáveis externas ou interações com o usuário**.
 - A função não deve imprimir nada (**a menos que seja sua atribuição**). O teste da função com printf, scanf ou main() é feito à parte.
 - Seja direto. A função deve ser concisa, clara e focada. **Evite colocar dentro dela partes do código que pertencem à lógica do programa principal**.
 - **Use o main() apenas como espaço de testes**: você pode escrever diversos printf ali para verificar se a função está funcionando como deveria.
- Ao pensar cuidadosamente sobre os **parâmetros** e o **retorno da função**, você estará praticando um dos aspectos mais importantes da programação: **abstração**.